

Software Requirements Specification

for

ToGather Wedding Invitation App

Version 1.0 approved

**Prepared by Kris Pritchett, Colten Mikulastik, Keiran Indraanei,
Christopher Nguyen, and Jesus Zambrano**

Mystic Stoa

date created: 2/5/2026

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction.....	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope.....	1
1.5 References	1
2. Overall Description	1
2.1 Product Perspective	1
2.2 Product Functions.....	2
2.3 User Classes and Characteristics	3
2.4 Operating Environment	3
2.5 Design and Implementation Constraints	4
2.6 User Documentation.....	4
2.7 Assumptions and Dependencies.....	4
3. External Interface Requirements	5
3.1 User Interfaces.....	5
3.2 Hardware Interfaces	5
3.3 Software Interfaces.....	6
3.4 Communications Interfaces.....	6
4. System Features	6
4.1 Login Page.....	6
4.2 Wedding Details Planning Form	7
4.3 Dashboard.....	8
5. Other Nonfunctional Requirements	9
5.1 Performance Requirements	9
5.2 Safety Requirements	9
5.3 Security Requirements	11
5.4 Software Quality Attributes	11
5.4.1 Usability.....	12
5.4.2 Reliability.....	12
5.4.3 Availability	13
5.4.4 Maintainability.....	13
5.4.5 Portability.....	13
5.4.6 Scalability	13
5.5 Business Rules.....	14
6. Other Requirements	15
Appendix A: Glossary.....	15
Appendix B: Analysis Models	17
Appendix C: To Be Determined List.....	17

Revision History

Name	Date	Reason For Changes	Version
Kris Pritchett	2/10/2026	Added information to section 4 System features	1.0

Colten Mikulastik	2/9/2026	Adding Introduction to Document	1.0
Keiran Indraanei	02/10/2026	Added information to section 3 subsections.	1.0
Jesus Zambrano	02/13/2026	Added Information to Section 5	
Kris Pritchett	2/15/2026	Edited Section 4	2.0

1. Introduction

1.1 Purpose

The ToGather project version one is a wedding invitation management web application. This project's scope is to manage the design, storage, and sending processes of a wedding invitation.

1.2 Document Conventions

This document attempts to convey the high-level requirements of our project.

1.3 Intended Audience and Reading Suggestions

This document's intended audience is for software developers and project managers. At the beginning of this document, the table of contents describes where specific requirements are listed. If the reader is attempting to access requirements, features, or other information, they should begin by indexing the related heading.

1.4 Product Scope

Create a Web application to facilitate the invitation of guests to a wedding. The web application will support design of invitations, allowing the wedded to use 2-d design software, music, and basic text editing. The web application will store user generated content on the platform. The web application will store the designed invitation along with the email addresses and user accounts for the guest users, following security best practices, holding a maximum of 400 guests. The web app will keep track of guest accounts. The invitation will gather information like meal requirements, plus one requirement, and other input depending on the type of invitation. The web app will record guest responses to this invitation and will store this information and display for the wedded on a dashboard.

The scope of our project is to:

- create a webapp that facilitates the invitation of guests to a wedding
- create a webapp that facilitates the accepting of invitations
- create a webapp that facilitates the accepting of RSVP
- create a webapp that allows for design of invitations 3

Out of scope:

- Development of native iOS or Android applications
- Large scale user support or customer service features
- Legal or copyright clearance for third party music or media
- Multi-language support beyond English

1.5 References

N/A

2. Overall Description

2.1 Product Perspective

The Wedding Invitation Web Application, ToGather, is a new standalone web-based system designed to help couples create, customize, and manage digital wedding invitations. The system is not a replacement of an existing product. It is a new platform built from scratch using React for the frontend and Firebase services for the backend.

The application operates through a client-server architecture. Users access the system through a web browser on desktop or mobile devices. The frontend is developed using React and communicates with Firebase services, including Authentication, Firestore database, Cloud Storage, Hosting, and Cloud Functions. The system also integrates external services such as Google Maps API for venue display, Calendar integration for event downloads, and an email service for sending notifications. The system contains two main components. The frontend handles the user interface, invitation builder, dashboard, and RSVP forms. The backend manages authentication, database storage, file uploads, secure access tokens, and email notifications. All communication between the client and server is secured using HTTPS.

The product is designed as a mobile first application to ensure guests can easily view and respond to invitations from their smartphones.

2.2 Product Functions

The ToGather Wedding Invitation Web Application provides the following major functions:

User Account Management

Couples can create accounts, log in, log out, and reset passwords. Users can manage their profile information and delete their accounts if needed.

Invitation Creation and Customization

Couples can enter wedding details such as names, dates, times, and venue information. They can choose from multiple design templates, customize colors and fonts, upload photos and music, write messages, and create sections such as Our Story and FAQ.

Invitation Publishing and Sharing

The system generates a unique shareable link for each wedding invitation. Couples can share the link directly or through social platforms. Secure one-time access tokens to protect private invitations.

Guest List Management

Couples can add, edit, and remove guests. Each guest can be assigned a plus one limit of up to three additional attendees. Guest data is stored securely in the database.

RSVP Management

Guests can respond by selecting attending or not attending. They can indicate the number of attendees, provide dietary restrictions, and leave a message. The system prevents duplicate responses and sends confirmation emails.

Dashboard and Analytics

Couples can view RSVP statistics including total invited guests, confirmed attendees, declined responses, and pending replies. The system stores RSVP data to support trend analysis and future improvements.

Integration Features

Guests can download a calendar file to add the event to their personal calendar. The invitation displays an interactive map of the venue location using Google Maps. The system shall allow guests to download an event calendar file (.ics) to add the wedding to their personal calendar.

Security and Privacy

All data is encrypted in transit and at rest. Firestore security rules restrict data access. Invitation links use secure tokens with expiration to prevent unauthorized viewing.

2.3 User Classes and Characteristics

Couples or the host of the event

Couples are the main users of the system. They create and manage wedding invitations, customize content, manage guest lists, and track RSVPs. Couples are expected to have moderate technical knowledge and be comfortable using web applications. They will use the system frequently during the initial setup period and moderately until the wedding date. Their main goal is to efficiently share information and track attendance.

Guests

Guests are secondary users. They receive an invitation link and use the system to view wedding details and submit RSVP responses. Guests may vary widely in age and technical ability. The interface must be simple and intuitive. Most guests are expected to access the invitation through a mobile device. Their interaction with the system is short term and focused on viewing information and submitting RSVP responses.

Couples are considered the highest priority user class because they manage all core system features. Guests are also important but use a smaller subset of system functions.

2.4 Operating Environment

The application operates in a web-based environment.

Client Side

The system runs on modern web browsers including Chrome 90 and above, Safari 14 and above, Firefox 88 and above, and Edge 90 and above. Mobile browsers on iOS and Android are supported. JavaScript and cookies must be enabled. The design supports screen widths as small as 320 pixels.

Server Side

The backend runs on Firebase infrastructure hosted on Google Cloud Platform. Cloud Functions use Node.js version 18 or above. Data is stored in Firebase Firestore and files are stored in Firebase Cloud Storage. Firebase Hosting delivers the web application using a content delivery network.

A stable internet connection is required. All communication uses HTTPS protocol.

2.5 Design and Implementation Constraints

The frontend must be developed using React.

The backend must use Firebase services including Authentication, Firestore, Storage, Hosting, and Cloud Functions.

The system must follow a mobile first design approach.

The application must support up to 400 guests per wedding event.

The project must be completed within 2 semesters following an agile sprint structure.

The system must comply with privacy regulations.

Firestore security rules must be implemented to ensure proper access control.

The team must prioritize maintainability and documentation since this is a capstone level project.

2.6 User Documentation

The following user documentation will be delivered with the Wedding Web Application:

- **README file:** A project README will be included in the GitHub repository. The README will provide an overview of the project, setup instructions, installation steps, environment configuration, and deployment instructions. It will also describe the technology stack and how to run the application locally.
- **User Guide:** A digital user guide will be created in PDF format. This guide will explain how couples can create an account, build an invitation, manage guests, and track RSVP responses. It will also describe how guests can access invitations and submit RSVP forms.
- **Quick Start Guide:** A short guide will be provided for new users. This guide will explain how to register, log in, and publish an invitation. It will focus on the most common tasks.

2.7 Assumptions and Dependencies

Assumptions:

- A1. Users have access to a modern web browser and reliable internet connection.
- A2. Couples will provide their own wedding content, including photos and venue details.
- A3. Guests must have a valid email address or mobile phone number to receive invitation links.
- A4. Firebase services will remain stable and available throughout development and deployment.

Dependencies:

- D1. The system depends on Firebase services for authentication, database storage, file storage, hosting, and server-side logic.
- D2. The system depends on Google Maps API for displaying venue location.
- D3. The system depends on an email service provider for sending RSVP confirmations and notifications.
- D4. The system depends on third party libraries such as React Router and Firebase SDK.
- D5. If any of these external services become unavailable, system functionality may be affected.

3. External Interface Requirements

3.1 User Interfaces

The ToGather Wedding Invitation application shall provide a guest-facing user interface that allows invited guests to view wedding invitations and submit RSVP responses through a mobile focused web browser. Guests shall access invitations via a unique URL provided by the host and shall not be required to create an account or install any application.

The invitation interface shall present key wedding information, including event details such as names, date, time, and venue, in a clear and visually organized layout. The interface shall be designed using a mobile first approach and shall adapt to various screen sizes, including smartphones, tablets, and desktop devices, to ensure readability and usability across platforms.

The system shall include an RSVP interface embedded within the invitation page that allows guests to indicate attendance status and submit required response information. Upon successful submission, the system shall provide a confirmation message indicating that the response has been recorded.

If a guest attempts to access an invalid or expired invitation link, the system shall display a clear and informative error message explaining that the invitation cannot be accessed.

Detailed visual design specifications and layout diagrams shall be documented separately in a user interface design specification.

3.2 Hardware Interfaces

The ToGather application shall operate on standard consumer hardware capable of running modern web browsers. Supported devices include desktop computers, laptops, tablets, and smartphones. No specialized hardware components or peripheral devices are required to access or use the system.

3.3 Software Interfaces

The ToGather application shall interface with backend services responsible for invitation data retrieval and RSVP response processing. The user facing frontend shall communicate with backend components to retrieve invitation information based on the unique invitation URL and to submit RSVP responses entered by guests.

The system shall interact with a backend data storage solution to store and retrieve invitation and RSVP data. All interactions between the frontend and backend components shall occur through defined APIs.

3.4 Communications Interfaces

The ToGather application shall interface with backend services responsible for invitation data retrieval and RSVP response processing. The user facing frontend shall communicate with backend components to retrieve invitation information based on the unique invitation URL and to submit RSVP responses entered by guests.

The system shall interact with a backend data storage solution to store and retrieve invitation and RSVP data. All interactions between the frontend and backend components shall occur through defined APIs.

4. System Features

4.1 Login Page

4.1.1 Description and Priority

The ToGather Wedding App will require a login page for users to sign up and access the app. User data is expected to be stored in a database for login and logout purposes. The login page has high priority as it means that users need accounts for the app and that their data will be protected.

4.1.2 Stimulus/Response Sequences

- User clicks sign up.
- Sign up page displays for user information to be entered.
- Once entered and confirmed, data is then encrypted.
- After encryption, the data is sent to the database in its encrypted form.
- That data is now used as a key for future logins.
- After signing up, the user is redirected to the login page.
- The user logs in and is able to access the application.

4.1.3 Functional Requirements

REQ-1: The system allows the user to sign up.

REQ-2: The system allows the user to login.

REQ-3: The system allows the user to logout.

REQ-4: The system provides password resets.

REQ-5: The system displays all icons and textboxes.

Error Handling

Anticipated errors include incorrect inputs of user credentials when logging in, signing up, or resetting passwords. Handling errors with error messages and asking for the correct credentials in the correct format.

4.2 Wedding Details Planning Form

4.2.1 Description and Priority

The ToGather App will require a wedding planning form for the creation of the wedding event. This form is for the host, or couple, to design their outline for the wedding invitation, such as the venue, names of the bride and groom, and date/time.

4.2.2 Stimulus/Response Sequences

- User accesses the wedding builder page after clicking its related button.
- The page for the planner displays.
- Information for the wedding will be entered by the user.
- After completing, the information will be verified and saved.
- The page redirects to a completed view of the planner.
- Form validation after completion.
- User edits the details.
- Values are updated in the form and saved in the database.

4.2.3 Functional Requirements

- REQ-1: Button that redirects to the form.
- REQ-2: The system displays the planning form.
- REQ-3: The system allows viewing.
- REQ-4: The system allows data to be deleted.
- REQ-5: The system allows data to be edited and updated.
- REQ-6: The system allows viewing of the form.
- REQ-7: The system displays error messages.

Error Handling

Anticipated errors include, but not limited to, incorrect formatted information in a textbox, the form is not completed, incorrect form redirections, timeouts, etc. These errors will need to be tested for and dealt with as they crop up. Error messages and addition of guiding information may be necessary for proper form completion.

4.3 Dashboard

4.3.1 Description and Priority

The ToGather Wedding App requires a dashboard to be implemented, which will allow for ease of use, simplicity, and ease of access. The priority for the feature is high as it will allow for seamless operation of the site.

4.3.2 Stimulus/Response Sequences

- User accesses dashboard after logging in.
- User clicks on any accessible item on the page.
- User is redirected to the desired page.
- User may redirect back to the dashboard through a button.
- The system directs the user securely to each section of the application.

4.3.3 Functional Requirements

- REQ-1: The system redirects to the dashboard after logging in.
- REQ-2: The system displays RSVP, guests, etc. on the dashboard.
- REQ-3: The system displays menu options and dropdowns.
- REQ-4: The system allows the user to log out from the dashboard.
- REQ-5: The system icons are visible.

Error Handling

Anticipated errors include, but not limited to, crashing, redirection errors, data not displaying, dashboard not functioning correctly, no internet, etc. Functional errors cleared up with good connection between the front and back end of the application. Stable internet is required for connection to the site.

4.4 Invitation Customization and Preview

4.4.1 Description and Priority

The ToGather Wedding app requires a feature for generating invitations. This feature will allow couples to design their invitation and preview it. The priority is high since this is the main user experience for the app.

4.4.2 Stimulus/Response Sequences

- User navigates to the invitation page.
- System opens the invitation customization page.
- User creates and designs a new invitation.
- System updates the invitation with the custom build.
- The invitation is displayed with information about the wedding.
- User saves the invitation.
- System displays the complete invitation.
- The user previews the invitation.
- User edits the invitation.
- Invitation updates with new inputs.

4.4.3 Functional Requirements

REQ-1: The system shows the templates for the invitation.

REQ-2: The system allows for different font styles and sizes.

REQ-3: The system allows the user to create a new invitation.

REQ-4: The system allows the user to design the invitation.

REQ-5: The system allows the user to save the invitation.

REQ-6: The system allows the user to edit a completed invitation.

REQ-7: The system allows the user to preview the invitation during and after invitation creation and design.

Error Handling

Anticipated errors include, but not limited to, crashing, redirection errors, templates not displaying, design not updating, timeouts, editing not possible, preview not displaying, etc. Error messages will need to be provided for each situation that is not constrained by outages. The user should be able to go back to the dashboard when encountering an error.

4.5 RSVP System

4.5.1 Description and Priority

The ToGather Wedding App requires a RSVP form or system that will allow guests to confirm attendance at the wedding. The priority is high as this is a core requirement after invitations are sent out.

4.5.2 Stimulus/Response Sequences

- The user selects RSVP.
- RSVP form is displayed.
- The user enters their information.
- The user selects confirm for attendance.
- The user decides additional guests to bring.

- The user saves the form.
- The form is complete and returns to view the complete form.

4.5.3 Functional Requirements

- REQ-1: The system displays the form.
- REQ-2: The system allows the user to provide attendance status.
- REQ-3: The system allows for extra guests, up to 3.
- REQ-4: The system allows guests to provide dietary restrictions.
- REQ-5: The system allows for special messages.
- REQ-6: The system will display confirmation after submission.

Error Handling:

Anticipated errors include, but not limited to, crashing, redirection errors, data not displaying, RSVP not functioning correctly, no internet, cannot confirm status, cannot provide extra guests, etc. Error messages to be displayed for each potential problem.

4.6 Calendar and Map

4.6.1 Description and Priority

The ToGather Wedding App requires a calendar and map to be included for reminders of when and where the wedding will be held. Directions are provided through the map. The priority is high as this allows for core functionality.

4.6.2 Stimulus/Response Sequences

- User accesses the calendar.
- The calendar is displayed as a functional calendar from Google, Outlook, etc.
- Wedding details are displayed when completing a prior form.
- The user downloads the information.
- The data is saved locally.
- The user redirects the map.
- The map is displayed with the location of the wedding.
- The user views the wedding location.

4.6.3 Functional Requirements

- REQ-1: The system allows the user to download calendar data.
- REQ-2: The system displays the calendar and map.
- REQ-3: The system allows the user to edit the information in the calendar.
- REQ-4: The system allows guests to view the calendar and map.
- REQ-5: The system updates the calendar and map automatically.

Error Handling:

Anticipated errors include, but not limited to, crashing, redirection errors, data not displaying, calendar and map data not updating, data not downloading, etc. Handle errors through providing the type of error through a message.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The ToGather Wedding Invitation system shall satisfy the following measurable performance requirements:

PERF-1:

The system shall load invitation pages within 5 **seconds** under standard conditions.

PERF-2:

The system shall process RSVP submissions and store responses within 4 **seconds** after user submits.

PERF-3:

The host dashboard shall render guest response data for up to 150 **guests** within 5 **seconds**.

PERF-4:

The system shall support at least **50 concurrent guest users** accessing invitation links without performance degradation.

PERF-5:

The system shall support a maximum of **400 guests per wedding event**, as defined in project scope.

PERF-6:

The system shall maintain availability of at least **66% uptime** during the active invitation period, excluding scheduled maintenance.

PERF-7:

The system shall send invitations to all guests within a maximum of five minutes.

Rationale:

The project charter emphasizes mobile-friendly access and RSVP tracking, which requires responsive performance under typical event conditions.

5.2 Safety Requirements

SAFE-1 (Data Loss Prevention):

The system shall prevent accidental loss of invitation and RSVP data by maintaining persistent storage and performing automatic incremental saving of edits. The system shall save changes immediately upon modification and/or at regular intervals (e.g., every 30–60 seconds) to ensure minimal data loss.

SAFE-2 (Confirmation for Destructive Actions):

The system shall require explicit confirmation before destructive actions, including deleting invitations, removing guest records, or clearing RSVP responses.

SAFE-3 (Backup and Recovery):

The system shall provide a recovery mechanism for critical data (invitations, guest list, RSVP responses) such as scheduled database backups or version history for invitation templates.

SAFE-4 (Input Validation and Safe Defaults):

The system shall validate all required RSVP fields (attendance, plus-one, meal selection) and prevent submission of incomplete or invalid forms. When optional fields are omitted, the system shall store safe default values (e.g., “Not Provided”).

SAFE-5 (Prevent Incorrect Guest Association):

The system shall ensure RSVP submissions are linked only to the correct guest record associated with the unique invitation token, preventing a guest from submitting responses for another guest.

SAFE-6 (Error Handling and User Messaging):

If a system failure occurs during RSVP submission (network failure, server error), the system shall display a clear message indicating whether the RSVP was recorded and provide a safe retry option without duplicating responses.

SAFE-7 (Invitation Link Validity):

If a guest accesses an invalid or expired invitation link, the system shall display a clear error message and shall not expose any event details or guest data.

SAFE-8 (Content Safety Constraints):

If invitation features include background music or media, the system shall prevent automatically playing audio at high volume by default and shall provide user controls (mute/pause) to avoid unintended disruption.

SAFE-9 (Audit/Traceability for Critical Changes):

The system shall record a minimal audit trail for key host actions (e.g., invitation sent, RSVP form changes, guest list changes) to support troubleshooting and reduce the risk of undetected mistakes.

Rationale:

The system's scope emphasizes invitation sending, mobile viewing, and RSVP tracking. Safety requirements ensure failures do not cause real-world harm such as missed invitations, incorrect RSVP counts, or lost guest information.

5.3 Security Requirements

Because the system stores personally identifiable information (PII), including guest names, emails, meal preferences, and attendance status, strong security controls are required.

SEC-1:

All communication between client and server shall use HTTPS with TLS encryption.

SEC-2:

Host accounts shall require authentication using secure credentials.

SEC-3:

Passwords shall be stored using salted cryptographic hashing.

SEC-4:

Guest access shall be controlled via unique, token-based invitation URLs.

SEC-5:

Invitation tokens shall be sufficiently randomized to prevent guessing attacks.

SEC-6:

Guest response data shall only be accessible to the corresponding wedding host.

SEC-7:

Administrative functions shall be restricted to authorized users only.

Rationale:

The professor explicitly identified PII handling as a non-functional concern. The system must therefore enforce privacy and controlled access.

5.4 Software Quality Attributes

The ToGather Wedding Invitation system shall satisfy the following quality attributes to ensure usability, reliability, maintainability, and overall product quality. These attributes are critical to both end users (wedding hosts and guests) and the development team.

5.4.1 Usability

QUAL-USE-1:

Guests shall be able to complete RSVP submission in no more than 5 interaction steps.

QUAL-USE-2:

The system shall provide clear, human-readable error messages for invalid or incomplete input.

QUAL-USE-3:

Invitation design tools shall provide intuitive template selection and text editing without requiring technical knowledge.

QUAL-USE-4:

No application installation shall be required for guests to access invitations.

Preference: Ease of use is prioritized over advanced customization complexity.

5.4.2 Reliability

QUAL-REL-1:

The system shall store RSVP responses persistently without data loss under normal operating conditions.

QUAL-REL-2:

The system shall prevent duplicate RSVP submissions for the same guest unless explicitly modified by the host.

QUAL-REL-3:

System errors shall not expose internal system information to users.

QUAL-REL-4:

The system shall maintain data integrity supporting a one-host-to-many-guests relationship.

5.4.3 Availability

QUAL-AVA-1:

The system shall maintain at least **99% availability** during the active invitation period.

QUAL-AVA-2:

Scheduled maintenance shall be communicated to hosts in advance.

5.4.4 Maintainability

QUAL-MNT-1:

The system shall follow modular architecture separating frontend, backend, and database components.

QUAL-MNT-2:

Code shall follow consistent naming conventions and documentation standards.

QUAL-MNT-3:

The system shall allow modification of invitation templates without altering RSVP processing logic.

5.4.5 Portability

QUAL-PORT-1:

The system shall operate on modern browsers including Chrome, Safari, Firefox, and Edge.

QUAL-PORT-2:

The system shall function on desktop, tablet, and smartphone devices.

5.4.6 Scalability

QUAL-SCAL-1:

The system shall support up to **400 guests per event** in Version 1.

QUAL-SCAL-2:

The system architecture shall allow future expansion beyond 400 guests without complete redesign.

QUAL-SCAL-3:

The database schema shall allow support for additional event types beyond weddings in future versions.

5.5 Business Rules

The following business rules govern how the ToGather system operates. These rules define role permissions, limitations, and operational principles.

BR-1:

Each wedding event shall be associated with one host account.

BR-2:

Each host may manage up to **400 guest invitations** per event in Version 1.

BR-3:

Each guest shall be associated with one unique invitation token.

BR-4:

Guests shall only access invitation information associated with their assigned token.

BR-5:

Only hosts may edit invitation design content.

BR-6:

Guests may submit only one RSVP per invitation unless editing functionality is explicitly enabled.

BR-7:

RSVP fields shall include, at minimum:

- Attendance status (Accept/Decline)
- Meal preference (if enabled)
- Plus-one status (if enabled)

BR-8:

The system shall not require guest account creation unless explicitly approved by project scope modification.

BR-9:

The system shall not include native iOS or Android applications, as these are outside project scope.

BR-10:

Guest data shall only be used for wedding invitation management purposes.

6. Other Requirements

Appendix A: Glossary

Client–Server Architecture

A distributed system model where the frontend (client) sends requests to a backend (server), which processes logic and returns data.

Frontend

The user-facing portion of the application (UI), typically running in the browser.

Backend

Server-side logic responsible for authentication, database operations, business logic, and integrations.

API (Application Programming Interface)

A defined interface that allows one software component to communicate with another, typically via HTTP requests in web apps.

Mobile-First Design

A design approach where the UI is built primarily for small screens first, then adapted for larger screens.

Content Delivery Network (CDN)

A distributed network of servers that cache and deliver web assets geographically closer to users to improve load speed.

Modular Architecture

A system structured into independent components that can be developed and maintained separately.

Schema (Database Schema)

The structure and organization of data in a database (collections, fields, relationships).

Firestore

A Backend-as-a-Service (BaaS) platform providing authentication, hosting, database, and serverless functions.

Firestore

Firestore's NoSQL document database used for structured application data storage

PII (Personally Identifiable Information)

Any data that can identify an individual (e.g., name, email, meal preference tied to identity).

HTTPS (Hypertext Transfer Protocol Secure)

Encrypted version of HTTP that protects data in transit.

TLS (Transport Layer Security)

The cryptographic protocol used by HTTPS to encrypt communication.

Encrypted in Transit

Data is encrypted while being transmitted over the network.

Encrypted at Rest

Data is encrypted while stored in a database or storage system.

Persistent Storage

Data remains saved even after system restarts.

Incremental Saving

Saving changes progressively rather than all at once.

Data Integrity

Assurance that stored data is accurate and consistent.

Duplicate Submission Prevention

Mechanism preventing multiple identical RSVP entries from the same guest.

Functional Requirements

Define what the system must do.

Nonfunctional Requirements

Define how well the system must perform (performance, security, reliability, etc.).

Scalability

Ability of the system to handle growth (e.g., more guests).

Responsive Layout

UI automatically adjusts to different screen sizes.

Protected Route

Frontend routing mechanism that restricts access to certain pages unless authenticated.

Form Validation

Process of verifying user input meets required format and completeness rules.

Appendix B: Analysis Models

See Clickup for Architecture for reference.

Appendix C: To Be Determined List

- 1. Feature for Calendar on webapp*
- 2. Feature create user accounts for guests*